

Distributed objects and object databases

M Machura and T Jebb

Due to the on-going shift from centralised to decentralised computing and a general trend towards the globalisation of commercial, economic and social activities, the information systems community has been faced with strong demands to develop distributed information systems (DISs) that span wide geographical areas. Three technologies have had significant impact on the new generation of DISs — distributed object technology, object database technology and Web technology. The paper examines two of these — DOT and ODBT. Each has advantages over the other, but more importantly they complement each other, and ways of using them together are explored.

1. Introduction

This paper examines the relationship between distributed object technology [1] and object database technology [2, 3]. It considers how their functionality overlaps, and more importantly where they complement each other in the context of building distributed information systems (DISs).

Object database technology (ODBT) attempts to meet the demands of new advanced database applications. ODBT, as defined by the Object Database Management Group (ODMG), transparently extends one or more existing object-oriented (OO) programming languages with DBMS capabilities [4]. It offers substantially enhanced modelling power, based on an OO model, over traditional database technologies.

In modern organisations DB applications are no longer designed and built in isolation. Distributed object technology (DOT), as exemplified by the Object Management Architecture [5] defined by the Object Management Group (OMG), has emerged as a key technology for implementing systems that integrate various distributed applications, including DB applications, into large-scale heterogeneous DISs. The generally accepted architectures for modern DISs are the multi-tier, client/server architectures that separate clients (user interfaces layer), shared applications (business logic layer), and database servers (data layer). To keep a consistent view of the enterprise, the critical information must continue to be centralised in large database servers.

ODBT so far has not made significant inroads into conventional business applications [6, 7], although the number of organisations who have seriously started using it is rapidly growing. It is believed that DOT, as an underlying

technology for large-scale DISs, will draw ODBT into the mainstream commercial database systems.

2. What is needed in a distributed information system?

The distributed object model is particularly appealing to DIS developers, as it enables them to model and build a DIS as a collection of co-operating process and data objects which are distributed across the IT infrastructure and which communicate by exchanging messages. In contrast to the sequential object model, where all objects are running in a single address space on a single computer, the distributed model enables objects to send messages to multiple objects concurrently and receive results asynchronously or synchronously. The distributed object model extends the client/server model in that an object can be both a service requester and service provider. Simon [8] summarises advantages of the distributed object model as follows:

- implementation detail is hidden and the use of objects is based on well-defined object interfaces,
- objects provide a consistent approach to handling a wide range of media types,
- objects provide the foundation for DIS integration through re-packaging existing functionality,
- an object is a natural unit of distribution.

In general, DISs are composed of three types of distributed components — processing components, data components and presentation components. DIS developers should be given freedom to optimise the placement of distributed components over the available IT infrastructure.

In order to facilitate the distribution of components, distributed IT infrastructures normally offer one or more of the following high-level services:

- distributed presentation services,
- distributed processing services,
- remote data access services,
- remote file access services,
- distributed data management services,
- distributed object management services.

These help to achieve a desirable property of DISs, namely that of distribution transparency, which simplifies the way DISs are designed, built and maintained.

A particularly important aspect of DIS design is the distribution of data, which enhances local autonomy of data processing and leads to greater security and availability of data, and can significantly improve performance. At the design level, three key approaches can be taken to manage the distribution of data — partitioning, replicating and caching. DBMSs and file systems are typically used to implement data distribution. The former provide access to remote DB systems using a remote data access service, with the latter providing access to files using a remote file access service.

Among the traditional data models, relational DBMSs are particularly well suited to supporting data distribution, because of the ability to partition data into simple table structures held at different sites, and the ability to provide a single user view independent of the way the data is partitioned. However, it can be argued that an object data model offers a better way of representing and handling clusters (partitions) of objects (data). It is therefore believed that distributed object management and ODBMSs are natural technologies for implementing a distributed storage model (particularly when complex datatypes are involved) and provide a more adequate IT infrastructure for the development of DISs.

Over the past ten years, a new computing technology based on a distributed object model, distributed object technology, has evolved. It offers an effective framework for the integration of distributed, heterogeneous computer applications; it also makes it possible to compose applications from generic distributed services. Undoubtedly, DOT will play a key role in the design and implementation of DISs. Unlike ODBT, which attempts to impose a particular data-centric view and provides a number of high-level tools to be used within a predefined development environment, DOT builds on the concept of openness. It gives greater flexibility in constructing DISs from both data processing and computation components. It

provides a glue necessary to include different data-processing models and technologies in a single DIS. It enables DISs to be built from both fine-grained and coarse-grained objects in a consistent way and with comparable ease.

The most important DOT standard at the moment is the OMG's Common Object Request Broker Architecture (CORBA) [9, 10].

3. Choosing between ODBT and DOT

ODBT and DOT aim to provide support for transparent access to remote, externally managed objects. ODBT enables client applications to request services of objects stored in remote DBs; DOT enables client applications to request services of remote server objects. Despite apparent similarities, the two technologies differ significantly in the modes of communication and support offered for remote object access.

The distinction between the two technologies is often summarised in the following way:

'The Object Request Broker (ORB) takes the request to the object(server), while the ODBMS takes the object to the request(client)'.

As far as the current generation of ODBMSs is concerned, this statement is, in general, true. However, some ODBMSs, called active ODBMSs, enable client applications to run object operations on the server remotely.

In order to choose between ODBT and DOT, it is more useful to understand the purpose behind each technology. ODBMSs manage persistent objects (including their implementations) and provide remote access to them; ORBs offer a framework for accessing remote objects and are not concerned with their actual implementation. In ODB applications, many clients interact with one or more DB servers, and the interaction is largely under the client's control; in ORB-based applications, objects can interchangeably play the roles of service requesters (clients) and service providers (servers), and the basic mode of communication is peer-to-peer, two-way communication. Thus the purpose of ODBT is to distribute, store and manipulate complete objects, while the purpose of DOT is to integrate computing resources seen as distributed objects.

The following general comments can be made on the choice between ODBT and DOT when DISs are built (see also Roy and Ewald [11]). ODBT is preferable when:

- objects are expected to outlive applications,
- objects are expected to have unique identifiers only within a single-vendor ODB application,

- applications require sophisticated data modelling techniques,
- applications are data-intensive, i.e. they manipulate fine-grained, highly interlinked objects with infrequent (if any) references to objects stored outside a database,
- the key requirement is to efficiently manage persistent objects using transaction, query and semantic integrity facilities (even though DOT is expected to provide support for those distributed services, performance-tuned ODBMSs will probably always be superior),
- applications require less flexibility in placing object data and behaviour across the network,
- the client/server model of communication is adequate,
- frequent modifications and extensions of the existing applications are not expected,
- run-time modifications of object definitions (DB schema) are not expected (some ODBMSs support dynamic schema modifications but they usually require that new schema implementations be available at link-time).

DOT is preferable when:

- objects are needed only as long as the applications are running,¹
- objects must have unique identifiers within a network-wide scope (multivendor ODB interoperability),
- collaborating applications, developed using different programming languages and operating systems, run on different platforms,
- applications are perceived as coarse-grained, resource-intensive objects with fewer interrelationships and less frequent interactions between them,
- the key requirement is to transparently access remote object services,
- applications require more flexibility in placing object data and behaviour across the network,
- different modes of communication are required,
- applications manipulate objects that have special use requirements and cannot be instantiated at the client location,
- applications handle single global object instances for security reasons,
- frequent modifications and extensions of the existing applications are expected,

- run-time modification of object definitions (object interfaces) are required (client applications need not be recompiled or relinked when server applications change).

4. Why combine ODBT and DOT

The discussion presented in the previous section points to a number of significant differences between ODBT and DOT. At the same time, it should be clear that the two technologies have a lot in common. The similarities, mainly derived from the underlying object model, make the technologies compatible, while the differences make them complementary. Thus the combination of the two can lead to an enhanced technology for DISs.

DIS developers expect support for distributed services for remote data access, and transparent communication between geographically dispersed, heterogeneous DIS components. Both ODBT and DOT fulfil those two general requirements. ODBT meets the first by providing a coherent, performance-tuned set of basic services for manipulating persistent data. DOT infrastructures also provide support for distributed services, but due to the underlying requirement of openness it is rather unlikely that DOT will be able to deliver a level of performance comparable to that achieved by ODBT.

DOT offers a vision of business objects that reside on different computers and collaborate to achieve a required business goal. The communication between business objects is largely transparent. DOT is also a desirable technology for the implementation of a middle-tier of the three-tier architecture. It can be argued therefore that, in comparison with ODBT, DOT gives greater flexibility in transparently distributing and using computational resources.

In general, ODBT and DOT seem to offset each other's deficiencies rather than creating unnecessary redundancy [12]. When used together, they offer:

- complementary data transfer modes by enabling operations to be carried out at the client or server locations,
- uniform treatment of transient and persistent objects (current ODBMSs make a distinction between persistent and transient objects, e.g. the transaction or event notification services apply only to persistent objects),
- management of both code and data (most ODBMSs only manage the state of objects, whereas DOT provides support for managing the code that implements the objects),

¹ Note, however, that distributed objects can be made persistent if required.

- front-ends to legacy DB systems by hiding the existing applications behind standard distributed object interfaces.

The remainder of this section discusses how the compatibility and the consequent synergy between the two technologies are achieved.

4.1 Common object model

The ODMG-93 object model [4] is a superset of the OMG Core Object Model (on which CORBA is based). Figure 1 below shows the structure of the OMG object model which is broken into a set of components, with a distinguished core component defining the basic elements of the model, i.e. objects and operations.

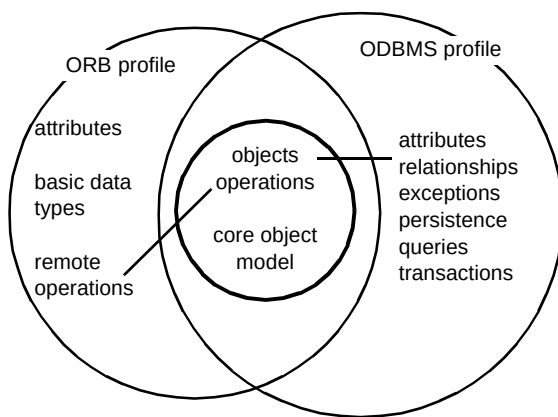


Fig 1 OMG's object model structure.

Each technology domain is expected to define a suitable object model by creating a so-called profile. A profile is a set of OMG components that must contain the core component. A given technology domain provides its own OMG-compatible extensions built around the core object model. The ODMG-93 profile, for example, includes support for persistent objects, properties (attributes and relationships), exceptions (the OMG core object model does not define exception returns on operations), queries and transactions.

In addition, the ODMG has proposed a number of candidate components that correspond to the common object services identified in the OMG Object Management Architecture (e.g. persistence and queries). The ODMG-93 standard for ODBs has clearly positioned itself within a broader and more flexible OMG architecture for distributed object systems. It also explicitly considers the integration of ODBT with CORBA. The purpose of this integration is to be able to treat objects stored in ODBs as CORBA objects, and to enable DB objects to transparently access CORBA objects.

4.2 Design flexibility

Probably the biggest benefit from combining ODBT and DOT is an increase in flexibility with which DISs can be designed and built. The designer can make several choices:

- between tighter integration of distributed services and the consequent higher performance offered by ODBT, and looser integration of services and greater flexibility and interoperability offered by DOT,
- between (usually) limited support for computer platforms offered by ODBT vendors, and built-in support for openness and heterogeneity of computing resources offered by DOT,
- between greater flexibility in partitioning/replicating/caching data offered by DOT, and higher level, but less flexible, distribution tools offered by ODBT,
- between narrower scope of object identity offered by an ODBMS vendor (single name space), and unrestricted scope of object identity offered by DOT (global name space).

4.3 Scalability

Scalability can be viewed as a special type of transparency. It is expected to minimise (ideally hide) the impact of the changes made to the existing DISs in terms of increased user and application demand (size, performance, platforms, distribution, etc). Distributed object systems scale better than ODB systems because of the underlying support for openness.

In comparison with DOT, ODBT can provide higher level scaling mechanisms without causing any disruption to application code. For example, some ODBMSs allow objects to be moved across DBs located at different sites; other ODBMSs support transparent archiving.

Support for distributed transactions and queries can also be regarded as a scaling facility. However, one of the major obstacles to achieving higher-level scalability in many ODB systems is the fact that programming languages such as C++ and Smalltalk are used, rather than higher level tools typically offered by relational database technology.

An interesting observation from the DIS perspective is that the integration of DOT and ODBT offers an additional type of scalability, i.e. that resulting from the compatibility of the underlying object models.

4.4 ORB-based middleware

Multivendor, heterogeneous DISs are inherently more difficult to build, secure and manage than traditional single-vendor centralised ISs. ODB systems will seldom be built in

isolation, perhaps with the exception of clearly defined, narrow application domains with homogeneous user requirements. Although three-tier DIS architectures can be designed using an ODBMS to provide the middle-tier functionality, future heterogeneous DISs will require a more open and flexible IT distribution infrastructure.

Different types of middleware have evolved over the years — remote procedure calls, message-oriented middleware, remote data access, transaction processing monitors and ORBs. The most promising technology, due to its support for distribution transparency and generic services such as distributed transactions, is ORB-based middleware.

DOT has the potential to become a key technology for future DISs, but the current generation of ORBs is still too fragile to be used in robust large-scale distributed applications. The three major, as yet unfulfilled requirements are:

- support for large volumes of transactions and objects,
- support for message queuing on both the client and server sides in addition to the existing support for object call processing,
- support for security.

The recent extensions of the OMG standard, such as Object Transaction Service and CORBAsecurity, provide a good basis for building a new generation of ORBs.

4.5 Standards

The emergence of standards is evidence that a software technology has reached a high level of acceptance and maturity. In the area of DISs, standards are necessary to ensure application portability and interoperability. Both ODBT and DOT have delivered *de facto* standards — the ODMG-93 standard and the OMG standard, respectively. The ODMG-93 standard is intentionally based on the OMG standard. Both standards are still undergoing gradual evolution. They are being improved and extended, often influencing each other.

Another standard of great importance for future DISs is SQL3 [13]. The corresponding standardisation bodies have established working relationships, and it is likely that the three standards will become compatible or even merge into one cohesive standard. Whatever the outcome of these efforts, they will certainly benefit the combined use of ODBT and DOT.

5. Combined use scenarios

The previous section provided a justification for the combined use of ODBT and DOT and briefly discussed expected benefits. The purpose of this section is to present a number of different scenarios for the combined use of both technologies (see also Appelbaum [14] and Baker [15]). The discussed scenarios fall within the two extremes of using ODBT and DOT separately — ODBT to develop conventional client/server DB applications, and DOT to develop open distributed systems.

Excluded from the presentation are two rather obvious scenarios:

- using CORBA services [16], such as naming and events, directly in an ODB application,
- accessing remote CORBA objects from within an ODB application which acts as a CORBA client.

A special case of the latter is when a remote CORBA object represents another ODB application. Those two scenarios simply describe the normal way of using CORBA to build distributed systems.

A more interesting question is how to make CORBA objects persistent and how to manipulate them in a consistent and safe manner. Seen in this light, the first three scenarios presented below, i.e. front-ending of DB objects, using the Object Database Adapter and using the Persistent Object Service, play a key role in the design of multi-tier DIS architectures. In all those scenarios, DOT provides the underlying communications infrastructure and expands the range of platforms which can access objects stored in ODBs. The fourth scenario illustrates how the Object Management Architecture and its common distributed services can be used to build an open ODB management facility which effectively acts as an ODBMS.

A clarification of some terms used in this section is necessary. Both technologies deal with the distribution of computing resources, so a distinction must be made between ODB clients/servers and CORBA clients/servers. Clients and servers are not to be understood here as network nodes, they are rather processes that run on node machines. Thus a CORBA client invokes CORBA objects in a local or remote CORBA server, and an ODB client activates DB objects, i.e. DB objects are loaded by an ODB server into the ODB client's cache. When the two technologies are used together in a multi-tier DIS, the middle tier will run processes (application servers) which are CORBA servers from the DOT perspective and ODB clients from the ODBT perspective.

5.1 Front-ending DB objects

ORBs can be used to provide connectivity between client applications and ODBs. The client application is implemented as a CORBA client which uses remote DB services. In this case, the developer builds a CORBA server and defines interfaces through which users can access the ODB (see Fig 2). The CORBA server (ODB client) has direct access to objects loaded from the ODB into its cache and is fully responsible for starting, committing and aborting ODB transactions. Due to CORBA's support for lightweight objects, a large number of ODB objects can exist in a single server process.

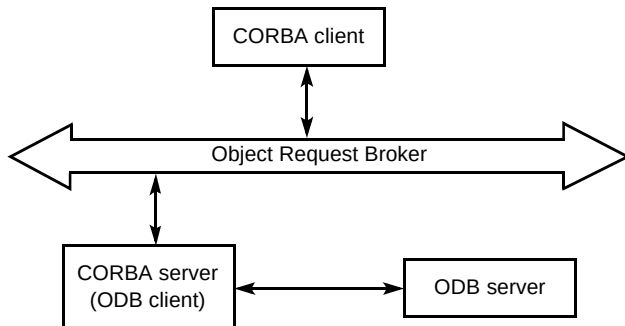


Fig 2 Front-ending DB objects.

Making DB objects visible to client applications in this way is called front-ending. What is effectively achieved here is a separation of client applications and ODB servers. Different levels of visibility of DB objects are possible, ranging from all DB objects to only selected high-level objects being visible to client applications. When building such client applications (CORBA clients), no knowledge of the DB schemas is required, as the ODB services are only accessed through predefined interfaces.

Using CORBA to separate client applications from ODBs offers several important advantages:

- CORBA clients can run on smaller machines,
- DB objects are manipulated by the CORBA server, so there is no need to load them all on to the client's machine — this feature may be particularly useful when mobility of clients' machines is required,
- slower (cheaper) communication links can be used to transmit object requests between CORBA clients and CORBA servers,
- an additional layer of security can be built into CORBA servers to control access to DB objects.

The front-ending technique can also be used to provide network access to existing (legacy) applications. It requires that the applications be presented to clients as CORBA objects. This is achieved by defining CORBA-compliant

interfaces behind which the existing applications are hidden (a process which is usually referred to as wrapping).

5.2 Accessing DB objects through an Object Database Adapter

All CORBA objects offering remote services must be individually registered with the ORB using a so-called Basic Object Adapter (BOA). Whereas this approach can be adequate for coarse-grained objects, large quantities of fine-grained objects stored in an ODB require a different type of object adapter, an Object Database Adapter (ODA) (see Fig 3). The purpose of an ODA is to ensure that fine-grained DB objects do not have to be registered individually with the ORB and that they are used directly by the client applications. Unlike the BOA, an ODA will not register individual objects in the CORBA implementation repository, nor will it invoke the requested services. Instead, it will pass the requests to the DB objects 'registered' with the ODBMS.

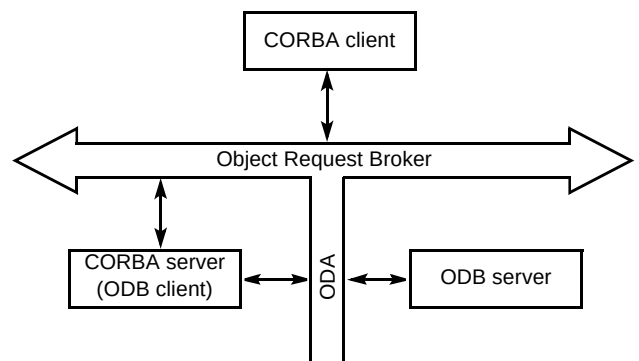


Fig 3 Using an Object Database Adapter.

Whereas the front-ending technique requires that the CORBA servers explicitly retrieve and manipulate DB objects, the ODA offers transparent access to objects stored in an ODB by presenting DB objects to client applications as CORBA objects. Unlike the front-ending method, the ODA approach requires that the client applications have knowledge of the ODB schema they use, i.e. the public components of class specifications as defined by the corresponding IDL representations.

Unless a standard specification of an ODA is agreed upon, each ODBMS will have to be equipped with several ODAs to make CORBA objects persistent for different ORBs. In fact, many ORB vendors develop ODAs in collaboration with ODBMS vendors (for example, IONA ships ODAs for ObjectStore and Versant). The object adapter-based, CORBA-compliant access to data stored in DBs is not restricted to ODBs. For example, if suitable mappings between data models are available, object adapters can also be built for data stored in relational DBs.

When developing a special object adapter such as an ODA, the CORBA vendor can either replace or more usually enhance the existing BOA.

Ideally, persistent CORBA objects should appear exactly like ordinary DB objects. There are, however, several reasons why an ODA should not store complete CORBA objects in the ODB [17]:

- it would be wasteful to keep ORB-specific definitions for all persistent objects,
- the frequently used CORBA operations 'duplicate' and 'release' update the object's reference count — these operations would have to be enclosed within DB transactions, adding to the overall processing time,
- whenever the constructor of a CORBA object is used, a new entry is added to a table of active objects kept by the ORB for a given process; in ODBs, however, the constructor of a persistent object is called only when an object is created and added to the DB.

In addition to storing CORBA objects, the ODA should also provide persistence to the corresponding object references. In this way, client applications can retain object references for later use. Also, by storing object references, permanent relationships between CORBA objects implemented on different servers can be expressed and maintained (a feature particularly important in multi-database systems).

Ideally, object references would be stored transparently in order to avoid explicit conversion between ORB-dependent object references and their corresponding string equivalents stored in the DB.

Several approaches to storing object references are possible [17]:

- pseudo-persistence,
- smart pointer-based persistence,
- virtual persistence.

The pseudo-persistence approach makes object references persistent, but not transparently storable. The smart pointer-based approach is similar to the pseudo-persistence approach, but in this case CORBA references are transparently storable. This is achieved by storing a CORBA reference as an instance of a smart-pointer class which automatically constructs a TIE object¹ when the smart pointer is dereferenced. Like smart-pointer persistence, virtual persistence also supports transparent storability of CORBA references. However, this approach

¹ The TIE mechanism is an optional object implementation strategy introduced in the CORBA C++ binding specification to 'tie' existing classes with CORBA object implementation classes.

can only be used with ODBMSs based on the page-server architecture. A CORBA reference is stored as a persistent pointer to an ORB-independent representative of the CORBA object. The representative is swizzled into the CORBA object at page-fault time. Performance evaluation conducted by Reverbel [17] points to pseudo-persistence as the best performer, the promising performance of smart pointer-based persistence, and the performance penalty paid with virtual persistence.

In general, using an ODA offers the following advantages:

- DB objects can be accessed like any other distributed objects registered with the ORB, i.e. the client application does not need to know whether objects are persistent or not,
- high-speed, transparent, distributed access to fine-grained application objects is supported, i.e. subspaces of object identifiers can be registered with the ORB rather than individual objects stored in the ODB,
- greater design flexibility is achieved in that designers can choose which objects are known to the ORB and which are not,
- distributed objects managed by the ODBMS can be transparently cached, moved or replicated to provide faster access.

5.3 Using an ODBMS to implement the Persistent Object Service

One of the standard CORBA services is the persistent object service [18]. It offers an alternative approach to making distributed objects persistent. The Persistent Object Service (POS) can be implemented using a variety of data storage techniques, but ODBT seems to offer the best match (see Fig 4).

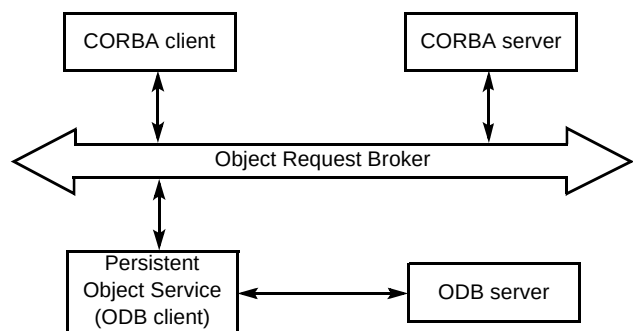


Fig 4 Implementing the Object Persistence Service.

POS allows objects to be responsible for managing their own persistent state. It offers a single object interface to multiple data stores and is designed to provide client control over object persistence. It supports both single-level and

two-level store models. Thus for some client applications, the persistence mechanism can be completely transparent. However, the main strength of POS lies in its support for a variety of corporate data stores which require a two-level store model.

An object can be made persistent by inheriting the PO (Persistent Object) class behaviour provided by POS. An object is responsible for storing and restoring its state. In particular, a persistent object inherits from the PO class (or provides) a facility to externalise its persistent state through a so-called protocol, i.e. a mapping between object and data store views of data. A request to externalise an object's state is issued by the underlying storage mechanism. A client application can use the persistence mechanism transparently, or it can control an object's persistence through the PO interface.

Every persistent object has a persistent identifier (an instance of class PID) which enables the object to be located within a data store. Client applications use persistent identifiers to access the persistent state of objects.

The POs provide the client's view of POS. The remaining elements of POS define components used by PO implementations on the server side. The Persistent Object Manager defines a uniform interface for the implementation of object persistence and insulates persistent objects from the Persistent Data Services. It defines five operations (connect, disconnect, store, restore and delete) that take persistent identifiers as parameters and enable clients to control persistence of POs. The Persistent Object Manager routes these operations from POs to the correct Persistent Data Services.

Persistent Data Services provide uniform interfaces for different data stores. They physically move data between persistent objects and data stores using protocols. POS currently defines three protocols:

- Direct Attribute (DA),
- Object Database Management Group (ODMG-93),
- Dynamic Data Object (DDO).

The DA Protocol was originally proposed by SunSoft and is based on a single-level store model. It enables an object implementor to directly access the store using a Data Definition Language that is a subset of IDL (attributes only). The ODMG-93 Protocol enables an object implementor to access an ODMG-93 database using the ODMG-93 Data Definition Language and the ODMG-93 language bindings. Finally, the DDO Protocol provides a data store/neutral representation of an object's persistent state. Using a special DDO data structure, an object implementor can dynamically describe an object's state without having to use IDL. In this approach, the Persistent

Data Service uses the object to store/retrieve data, instead of the object using the Persistent Data Service (as in the DA protocol).

A data store provides a storage mechanism for object's data. POS also contains IDL definitions that represent encapsulations of the X/Open Callable Level Interface (CLI). CLI is a standard for accessing SQL-based DBs and combines Microsoft's ODBC and Borland's IDAPI.

5.4 POS or ODA?

An ODA is based on a single-level store model and offers tighter ORB/ODBMS integration. POS is designed to provide support for storing objects in single-level and two-level stores, allowing for different types of persistent storage to be used. POS is therefore less restrictive than an ODA. An ODA seems to be more suitable if distributed objects are to be kept in an ODB.

It can be argued, however, that an ODA can also be used to build heterogeneous DISs, providing the necessary mappings between different data models are available. An example of such an approach is the TeleMed project [19] undertaken at Los Alamos National Laboratory. Using object-relational mapping and an ODA, the project integrates relational DBMSs into the ORB environment. TeleMed is a heterogeneous multidatabase system that allows data kept in local data stores to be seen through a unified OO global schema. All persistent data, independent of the underlying data model, can be accessed and manipulated through a standard C++ interface and an OO version of SQL.

IBM, one of the two key proponents of POS, maintains that a two-level store model is more suitable for the construction of open DISs. However, as recent experiments have shown, an ODA based on a single-level store model can also be used in open DISs. Reverbel [17] even believes that 'object-relational mapping plus an Object Database Adapter is the best way to integrate relational DBMSs into the ORB environment.' Interestingly enough, the persistent service offered within IBM's Open Blueprint product line features support for a single-level store model as well.

5.5 Building an object database management facility from CORBAServices

An ODBMS can be decomposed into a number of basic distributed services such as persistence, transactions, concurrency, recovery, versioning and security. A set of such services is carefully designed and integrated by each ODBMS vendor to meet high-performance requirements of DB applications. In contrast to ODBMSs, the Object Management Architecture offers a much larger set of standard distributed services necessary to build open

distributed systems. The ODBMS services constitute a (specialised) subset of CORBA services.

The basic distributed services (CORBA services) are not only used as building blocks to directly develop distributed applications, but also to develop a variety of standard higher level distributed facilities, called CORBA facilities [20]. Thus, in theory, it should also be possible to build a common ODB management facility which would provide services expected of ODBMSs (see Fig 5). Whether such an 'open system' approach is practical remains to be seen. Firstly and foremost, the overall performance may prove unacceptable. Secondly, there is an obvious difficulty if some of the services (e.g. persistence) have already been implemented using an existing ODBMS. It seems therefore that special-purpose (rather than general-purpose) ODB management facilities of this kind may be built in the future.

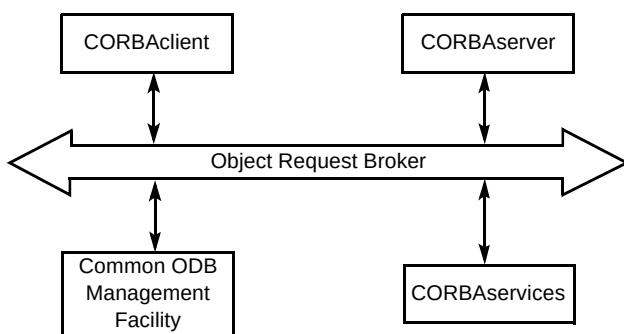


Fig 5 Creating an object database management facility.

6. Some important facilities

There are a number of fundamental facilities offered by traditional databases. Among the most important are support for transactions, queries, and relationships between data.

6.1 Transactions

A design issue closely linked with persistence is support for transactions. In DB systems, all access to DB objects takes place within transactions. An ODA must therefore also enable the CORBA server to start, commit and abort sequences of operations on DB objects.

In the simplest case, every remote operation is treated as a separate transaction. This normally leads to a large overhead associated with transaction processing. An ODA should therefore enable the developer to request several remote operations in a single transaction.

The concept of transactions is regarded as the single most important contribution to reliable distributed computing. It originated in DBMSs to support secure multi-user access to shared data resources. It was subsequently used and extended by TP (transaction processing) monitors to

ensure reliable data processing in large-scale DISs. It is a foundation for distributed computing in general.

Transactions, understood as units of work in DB applications, are characterised by the following properties (known as ACID properties):

- atomicity — a transaction is indivisible, and it is either carried out in its entirety or it is not carried out at all,
- consistency — a transaction transforms the DB from one consistent state to another consistent state,
- isolation — a transaction is carried out independently of other concurrent transactions, and its partial results are not visible to other transactions until it successfully completes (commits),
- durability — the result of a committed transaction is permanently recorded in the DB.

DB transactions, by definition, must have all the four ACID properties. TP monitors provide support for atomicity, consistency and isolation of distributed transactions in DISs; the durability property is supported by DIS data stores. DBMSs and TP monitors offer a collection of distributed services that are closely integrated and optimised for performance.

CORBA's Object Transaction Service (OTS) offers support for atomicity and consistency, but not for isolation and durability [21]. To this end, OTS can be used together with other CORBA services, i.e. the Object Concurrency Service and Persistent Object Service, respectively.

The basic DBMS distribution services (i.e. transactions, concurrency and querying) typically involve persistent objects only. The ODMG-93 standard specifically makes a distinction between transient and persistent objects and states that (at least in its current version) aborted transactions are not expected to restore transient object states. By contrast, the CORBA OTS, Concurrency Control Service and Object Query Service apply to both persistent and transient objects.

ORBs and TP monitors

TP monitors constitute a layer between client applications and DB servers. They play the role of external transaction managers, handling large numbers of clients, routing transactions across DB systems and load-balancing their execution. A typical scenario involves a distributed transaction which is started by a client application. It is divided into a sequence of subtransactions that are carried out on different DB servers and controlled by a TP monitor. TP monitors, commonly used by organisations running mainframe DBs, have not had much impact on client/server DB applications, mainly because, in comparison with other

vendors of client/server software, they were slow to provide adequate support for the new computing paradigm.

ORBs, extended with the CORBA transaction and concurrency services, are destined to become next-generation TP monitors for open distributed systems. An ORB, or more precisely its object adapter, manages concurrent requests for object services. In order to activate objects, the object adapter creates new processes, starts new threads in existing processes, or reuses some of the existing threads or processes. It plays a role similar to that played by a TP monitor, 'funneling' large numbers of incoming client requests into a much smaller number of processes at the DB server, thereby relieving the operating system from having to provide a dedicated process for each client request. However, the current generation of ORBs is not robust enough to support the integrity of transaction management.

The TP monitor vendors showed a lot of interest and actively contributed to OTS (and other related CORBA-services such as the Concurrency Control Service). Their experience in delivering robust solutions for large-scale commercial transaction processing systems makes them ideal partners for ORB vendors. The two types of vendor complement each other [22]. The strength of CORBA lies in its powerful mechanisms for standard activation of remote objects; the strength of TP monitors lies in their sophisticated mechanisms for process scheduling. If TP monitors are used to provide the scheduling function in future highly parallel object adapters, CORBA will become a viable solution for the construction of demanding OLTP applications.

In general, there are three basic ways of including TP monitors in the distributed object environment [23].

- The TP monitor provides an object layer that replaces the ORB

For TP monitor vendors, this is the preferred approach since it promotes the use of TP monitors as the underlying transactional middleware. A TP monitor is made available to the client object through an interface expressed in a proprietary interface definition language. In this way, the CORBA OTS is effectively replaced by services offered by the TP monitor. The obvious drawbacks of this approach are not only the loss of ORB interoperability but also the lack of many important common object services required in distributed systems.

- The ORB acts as a client to the TP monitor

This is probably the most frequently used approach. It enables the existing TP monitors to be included in the CORBA environment in a relatively straightforward manner. To this end, the TP monitor must be equipped with IDL-based interfaces. If the OTS interfaces are

supported, a relatively smooth integration with the open CORBA environment can be achieved. The drawback of this approach, however, is that transaction processing is under control of the TP monitor. From the client's point of view, the integrity of transaction management should be controlled by the ORB. In some implementations, CORBA servers (rather than CORBA clients) directly use the TP monitor facilities (e.g. Orbix — Tuxedo integration).

- The TP monitor is embedded in the ORB

This represents the preferred approach to combining the two technologies — the ORB provides the underlying distributed communication infrastructure, and the TP monitor is responsible for handling the integrity of large-volume transaction processing. An additional advantage is that the resulting middleware offers access to all standard CORBA services.

Object Transaction Service

For reliability reasons, concurrent manipulation of persistent data in DISs should be based on transactions. This requirement makes OTS the key component of the distributed infrastructure for CORBA-based DISs. OTS has the following features.

- Multiple transaction models are supported

OTS includes the flat (mandatory) and nested (optional) transaction models. The flat model is similar to the X/Open DTP (Distributed Transaction Processing) model. The increasingly important nested transaction model protects multi-operation transactions from network failures. Each operation is implemented within a separate subtransaction. If any operation cannot be completed, then only its corresponding subtransaction is aborted, not the containing parent transaction. In such cases, a remedial action can be undertaken without having to abort the high-level transaction.

- Both object and procedural applications can participate in the same transaction

OTS takes into account the existing procedural transaction standards, notably the X/Open DTP model. As a result, object transactions can interoperate with procedural transactions.

- Network interoperability is supported

Transactions are expected to span heterogeneous components of a DIS. A transaction service of one system must be able to co-operate with a transaction service of another system using different ORBs. In addition, a single ORB may support several transaction services. These requirements (among others) were taken into account in the CORBA 2.0 specification.

- Both system-managed and application-managed sharing of common transactions is supported

OTS provides different levels of managing transactions by client objects. In the simplest case, the client may use implicit OTS mechanisms; it can also take full control over the way common transactional resources are used.

- Objects can participate in a transaction

IDL interfaces of existing objects do not have to be changed when the objects are to participate in a transaction. A non-transactional object can be included in a transaction by simply making its IDL specification a subtype of an abstract OTS class.

Transactional clients and other transactional objects can collaborate by exchanging explicit method invocations, or they can rely on OTS to transparently (to a large extent) process transactions on their behalf. OTS provides a number of IDL interfaces to meet various control requirements of transactional clients, transactional servers and recoverable servers.

Apart from security, OTS is possibly the most important CORBA service. Combined with OTS, the ORB implements a TP monitor capability for open distributed systems. To achieve this goal, several OTS requirements had to be met in the CORBA 2.0 specification, e.g. additional standard exceptions, ORB-based management of the transactional context, and consistent transaction representation across heterogeneous ORBs. The full OTS can only be provided in a CORBA 2.0-compliant environment.

OTS also has a relatively large number of relationships with other CORBA services. For example, the Object Relationship Service can be used together with OTS to support referential integrity. To provide the atomicity and isolation properties, OTS can be used together with the Concurrency Service. Similarly, to provide the atomicity and durability properties, OTS can be used together with POS.

Concurrency Control Service

The Concurrency Control Service (CCS) is basically a locking system [24]. The CCS interfaces allow client objects to acquire and release locks, thereby protecting a shared resource from simultaneous access. Every shared resource must be protected by a separate collection of locks, called lockset. CCS defines several types of locks — read, write, intention read, intention write, and upgrade (the last type being used to prevent deadlocks).

CCS does not define a shared resource; it specifies interfaces that a resource must implement if it is to be accessed by many clients concurrently. Similarly, every

shared resource must implement a mechanism to create and manage a lockset.

CCS supports both transactional and non-transactional modes of operation. Both transactional and non-transactional clients can access the same resource. CCS is specifically designed to work with OTS. When in the transactional mode, CCS implicitly uses the transactional context associated with a client's thread. CCS is also designed to work with OTS nested transactions.

6.2 Queries

The Object Query Service (OQS) [25] provides support for query operations on collections of objects. The queries, expressed as declarative statements, enable arbitrary users and objects to select, insert, delete and update collections of objects. OQS has the following features:

- it allows the querying of any objects with arbitrary attributes and operations — objects can be of any granularity, and both persistent and transient objects can participate in the queries,
- it provides support for both associative and navigational queries,
- it maps well to the existing DB query mechanisms and allows use of methods for enhancing query performance, such as caching and indexing,
- it is based on standards such as SQL Query and OQL,
- it offers flexibility in implementation and extensions.

Based on a general model of predicate-based queries, OQS can accommodate a variety of query languages, including object derivatives of SQL, object query languages and graphical query languages. However, in order to provide interoperability between different query systems, OQS must support either SQL Query or OQL.

SQL-92 is the current, generally accepted standard in the relational DB world. SQL-92 Query is a subset of SQL-92 that incorporates only statements dealing directly with query. SQL-92 and its future versions are referred to as SQL Query. Similarly, OQL represents the current ODMG offering OQL-93 and its future versions. The two standardisation bodies behind SQL and OQL, i.e. the ANSI X3H2 Committee and the ODMG, have agreed to work towards a convergence between SQL Query and OQL.

It is expected that vendors will extend OQS and provide a variety of sophisticated query facilities.

6.3 Relationships

Objects are units of encapsulation, hiding internal data and behaviour implementation behind public interfaces. Objects deliver the expected functionality by forming associations (relationships) and collaborating (exchanging messages) with other objects. Examples of relationships are ownership, collection, and the commonly used reference and containment relationships. In general, relationships can be static and dynamic.

In non-distributed applications, relationships between objects are represented and manipulated in a rather *ad hoc* manner by means of object addresses. In distributed applications, a higher-level mechanism is required to manage relationships between remote objects. Another important requirement is that distributed objects should be able to create new relationships without having to change their definitions or recompile.

The purpose of the Object Relationship Service (ORS) is to support the creation of arbitrary dynamic relationships between immutable distributed objects [26]. The service makes it possible to keep track of continuously changing relationships between objects. It is therefore applicable in numerous situations, such as enforcing integrity constraints in DISs, assembling component suites for integration purposes, managing workflows, and manipulating distributed object containers. ORS is destined to play a crucial role in future distributed systems which increasingly will be built as sets of objects that form dynamic configurations, manage themselves and other computing resources, maintain continuous communication with other objects and users, and move across networks.

7. Conclusions

Open access to databases is vital in today's heterogeneous DIS environments. The integration of DOT and ODBT allows the construction of ORB-connected database systems. Combined with Web technology, future DISs will also offer universal user interfaces based on Java-enabled Web browsers.

At present ODBT is used in specialised database applications requiring support for complex data types and highly interrelated data. It also offers an integration framework for DISs that comprise different types of databases. So far, however, ODBT has not been accepted in mainstream business applications. ODBT is expected to gain in popularity when the ODMG-93 standard is universally supported by the object database vendors. It seems, however, that the best chance for ODBT to reach the commercial world of database applications lies in its synergistic use with DOT and Web technology.

DOT is treated in this paper as the underlying technology for DISs. It provides a framework for the integration of distributed, heterogeneous computer applications; it also makes it possible to compose applications from generic distributed services such as persistence, transactions, queries, relationships, security, etc. DOT provides the glue necessary to bind different data-processing models and technologies into a single DIS. It enables DISs to be built from both fine-grained and coarse-grained objects in a consistent way. Unlike current ODBT, it also allows persistent and transient objects to be treated uniformly. However, DOT has not come of age yet, and its two main standards, CORBA and Microsoft's DCOM, are still undergoing important extensions and improvements.

The paper contrasted DOT (as defined by the OMG) and ODBT (as defined by the ODMG), focusing on their differences, but emphasising mainly their compatibilities which contribute to increased design flexibility and scalability of DISs. It presented several important ways in which the two technologies can be used together — front-ending database objects, accessing database objects through an Object Database Adapter, using an ODBMS to implement the Persistent Object Service, and building an object database management facility from generic distributed services. Several fundamental distributed facilities, such as transaction processing, concurrency control, query processing and relationship maintenance, were also discussed in the context of DOT and ODBT.

Probably the single most important decision to be made when building a CORBA-based DIS is the choice of an ORB. Support offered by the ORB will have a significant impact on the development process and scalability of the resulting system. The ORB features to be considered include:

- full compliance with the OMG's Common Object Request Broker Architecture,
- support for the Internet Inter-ORB Protocol (IIOP) which is necessary for seamless integration and scalability,
- support for distributed transaction processing,
- support for security,
- a set of additional distributed services such as query processing, relationships, event notification, time, etc,
- support for multi-threading.

This paper is based on a fuller report, available within BT, at http://www.srd.bt.co.uk/projects/DIM/documents/bourne_mouth/bmth_report_2/contents.html. This contains more detailed discussion, and descriptions of programming experiments carried out to support the study.

References

- 1 Orfali R et al: 'The essential distributed objects survival guide', Wiley (1996).
- 2 Loomis M: 'Object databases', The Essentials, Addison-Wesley (1995).
- 3 Cattell R: 'Object data management: 'Object-Oriented and Extended Relational Database Systems'', (2nd Edition), Addison-Wesley (1995).
- 4 Cattell R: 'The object database standard: 'ODMG-93 (Release 1.1)', Morgan Kaufmann, <http://www.odmg.org> (1994).
- 5 'Object Management Architecture Guide', Revision 3.0, Wiley (1995).
- 6 Kim W: 'Object-oriented database systems: Promises, reality and future', in Kim W (Ed): 'Modern Database Systems: the object model, interoperability and beyond', Addison-Wesley, pp 255—280 (1995).
- 7 Kotes-Dittrich A and Dittrich K: 'Where object-oriented DBMSs should do better: a critique based on early experiences', in Kim W (Ed): 'Modern Database Systems: the object model, interoperability and beyond', Addison-Wesley, pp 238—254 (1995).
- 8 Simon E: 'Distributed information systems: From Client/Server to Distributed Multimedia', McGraw-Hill (1996).
- 9 CORBA: 'Common Object Request Broker Architecture and Specification', Version 2.0, Object Management Group (July 1995).
- 10 Siegel J et al: 'CORBA principles and programming', Wiley (1996).
- 11 Roy M and Ewald A: 'Should you use an ORB or an ODBMS?', Object Magazine, pp 74—76 (January 1994).
- 12 Reddy M: 'ORBs and ODBMSs: two complementary ways to distribute objects', Object Magazine, pp 24—31 (June 1995).
- 13 Melton J (Ed): 'Database language SQL3', ISO/ANSI Working Draft (1994).
- 14 Appelbaum R: 'An ORB plus an ODBMS', Object Magazine, pp 61—63 (September 1995).
- 15 Baker S: 'CORBA and databases: do you really need both?', Object Currents 1, No 5, SIGS Publications (May 1995).
- 16 'CORBAServices: Common Object Services Specification', Revised Edition, Object Management Group (March 1995).
- 17 Reverbel F: 'ORB/ODBMS integration in the sunrise project', Los Alamos National Laboratory, http://www.acl.lanl.gov/~reverbel/reverbel_orb_odbms.html (June 1996).
- 18 'Persistent Object Service', Object Management Group, <http://www.omg.org/docs/1994/94-10-07.ps> (1994).
- 19 'TeleMed Virtual Patient Record System, Chapter III: Future Developments', Los Alamos National Laboratory, <http://www.acl.lanl.gov/TeleMed/architecture.html> (April 1996).
- 20 'CORBAfacilities: Common Facilities Architecture', Version 4.0, Object Management Group (November 1995).
- 21 'Object Transaction Service', Object Management Group, <http://www.omg.org/docs/1994/94-08-04.ps> (1994).
- 22 Cobb E E: 'TP monitors and ORBs: a superior client/server alternative', Object Magazine, pp 57—61 (February 1995).
- 23 Boucher K D: 'ORBs and TP monitors: a marriage made in electronic commerce heaven', First Class (OMG's magazine), pp 12—13 (June/July 1996).
- 24 Concurrency Control Service, Object Management Group, <http://www.omg.org/docs/1994/94-05-08> (1994).
- 25 'Object Query Service', Object Management Group, <http://www.omg.org/docs/1995/95-01-01.ps> (1995).
- 26 'Object Relationship Service', Object Management Group, <http://www.omg.org/docs/1994/94-05-05.ps> (1994).



Marek Machura graduated from Technical University of Warsaw with an MSc in Control Engineering and Computing, and received a PhD in Computing from the same university in 1974. He worked as a researcher in the Institute of Atomic Energy, Warsaw, Stuttgart University, and Engineering Design Research Centre, Glasgow. He conducted research and led a number of R&D projects in the areas of numerical software, intelligent front-ends to evolving software systems, and design support environments. Currently he is a senior lecturer at Bournemouth University. His main research interests include distributed object computing and object databases. Currently he is investigating new approaches to distributed data management in self-organising information systems.



Tim Jebb joined BT in 1984. He initially worked in a group providing technical support for IBM mainframes, specialising in data communications. In 1986 he moved into a technical support role for the BT WatchMan network management product. From there he joined BT Laboratories, developing techniques for interworking between computers using proprietary protocols. He has recently been working on distributed information management, and information services, contributing to the BT Touchpoint service. He is currently developing Oracle software for transferring information between an Oracle system and an ObjectStore system.